Agenda

- Object Types
- Advanced Types
- MPA vs SPA
- React
- JSX

Object Type

1.Array

- An Array is a collection of multiple values stored in a single variable.
- In TypeScript, an array can be defined with a specific data type for its elements.
- All elements of an array generally follow the specified type.

```
let numbers: number[] = [10, 20, 30, 40];
let names: string[] = ["John", "Alice", "Bob"];

// Alternative Syntax
let numbers: Array<number> = [10, 20, 30];
let names: Array<string> = ["John", "Alice"];
```

2.Tuple

- A Tuple is a special type of array where:
    1. The number of elements is known.
    2. The type of each element is known.
    3. The order of the types is important.
- Its each position has a specific, predefined data type.
- JavaScript just sees them as regular mutable arrays at runtime
- TypeScript uses tuples at compile time to enforce strict structures—like ensuring the first element is always a string and the second is always a number.
- To define a tuple, pass an array of types instead of a single type:

```
let employee: [number, string, boolean];

// Initialization (Must match the exact order and length)
employee = [101, "Alice", true];

// Throws Type Error (wrong order)
// employee = ["Alice", 101, true];

// Throws Type Error (missing elements)
// employee = [101, "Alice"];
```

SUNBEAM INFOTECH

Named Tuple Elements (Highly Recommended)

- we can provide descriptive labels to each position in the tuple.
- This does not change the runtime behavior, but it drastically improves auto-complete tooltips inside IDE.

```
// Labels ('latitude' and 'longitude') make the tuple self-documenting
type GeoLocation = [latitude: number, longitude: number];

const officeLocation: GeoLocation = [18.5204, 73.8567];
```

3. Object

- An Object represents a collection of properties and their values.
- Each property can have its own data type.
- In TypeScript, we can define the structure of an object by specifying the property names and their types.

```
let student: {
    name: string;
    age: number;
    isActive: boolean;
} = {
    name: "John",
    // age: "25", // NOT OK
    age: 25,
    isActive: true
};
```

Optional Properties

- A property can be made optional using ?.

```
let student: {
    name: string;
    age: number;
    email?: string;
} = {
    name: "John",
    age: 25
};
```

Object Using type

- For reusable object structures, we normally define a type:

SUNBEAM INFOTECH

```ts
type Student = {
    name: string;
    age: number;
    isActive: boolean;
};

let student: Student = {
    name: "John",
    age: 25,
    isActive: true
};
```

Advanced Types

1. type alias

- A Type Alias allows us to give a name to an existing type or a custom type definition.
- It is created using the `type` keyword.
- Type aliases can represent:
    - Primitive types
    - Objects
    - Arrays
    - Tuples
    - Union types
    - Intersection types
    - Literal types
    - Function types

SUNBEAM INFOTECH

```ts
type User = {
    name: string;
    age: number;
};

const u1:User = {
    name:'anil,
    age:30
}

const u2:User = {
    name:'mukesh,
    age:32
}
```

2. union

- A Union Type allows a value to have one of multiple specified types.
- It is created using the pipe (`|`) operator.

- A union can be used with variables, function parameters, return types, properties, etc.
- The value can contain any one of the types specified in the union.
- TypeScript performs type checking to ensure that only the specified types are used.
- When working with a union, TypeScript may require type narrowing before performing type-specific operations.
- `Type narrowing` can be performed using techniques such as `typeof`, `instanceof`, equality checks, and custom type guards.
- Literal union types can restrict a value to a specific set of allowed values.

```
let id: string | number;
id = 101;       // OK
id = "A101";    // OK
// id = true;      // Error
```

```
function printValue(value: string | number) {
    console.log(value.toUpperCase()); //  Error
}
```

```
function printValue(value: string | number) {
    if (typeof value === "string") {
        console.log(value.toUpperCase());
    } else {
        console.log(value.toFixed(2));
    }
}
```

Interface

- An Interface defines the structure or contract of an object.
- It specifies the properties and methods that an object should have.
- Interfaces are created using the `interface` keyword.
- Properties can be required or optional.
- Properties can be marked as `readonly`.
- Interfaces can define methods and function properties.
- An interface can extend another interface using `extends`.
- A class can implement an interface using `implements`.
- Interfaces work well with Union Types and Type Narrowing.

```
interface User {
    name: string;
    age: number;
}
```

SUNBEAM INFOTECH

```
const user: User = {
    name: "John",
    age: 25
};
```

Optional Properties

- make a property optional using ?

```
interface User {
    name: string;
    age: number;
    email?: string;
}

const user1: User = {
    name: "John",
    age: 25
};
```

Interface vs Type Alias

- Both can describe objects.
- For basic object structures, they can look almost identical.

```
interface User {
    name: string;
    age: number;
}

type User = {
    name: string;
    age: number;
};
```

SUNBEAM INFOTECH

- A type alias can easily represent things that aren't object structures:

```
type ID = string | number;
type Status = "pending" | "success" | "failed";
```

- An interface is primarily designed for describing object/class contracts.

```
interface Person {
    name: string;
}
```

```
class Employee implements Person {
    employeeId: number;
}
```

Generics

- Generics allow us to write reusable code that works with different data types while still maintaining type safety.

```
function swap<T>(n1: T, n2: T): [T, T] {
    return [n2, n1]
}

// const [first, second] = swap(10, 20)
// const [first, second] = swap('sunbeam', 'infotech')
const [first, second] = swap<boolean>(false, true)
console.log(`${first} - ${second}`)

class Stack<T> {
    index: number = -1;
    arr: T[] = []

    push(e: T): void {
        this.index++
        this.arr[this.index] = e
    }
    pop(): void {
        if (this.index > -1)
            this.index--;
    }
    peek(): T | number {
        if (this.index != -1)
            return this.arr[this.index];
        else
            return -1
    }
}

// const s1 = new Stack<Number>()
// s1.push(10)
// s1.push(20)
// s1.push(30)

const s1 = new Stack<String>()
s1.push("sunbeam")
s1.push("pune")
s1.push("karad")
console.log(`Element at top - ${s1.peek()}`)
```

SUNBEAM INFOTECH

```
s1.pop()
console.log(`After pop, Element at top - ${s1.peek()}`)
```

Multi page application (MPA)

- the application which contains multiple pages designed
- it sends the request to the server every time for a page when data is requested
- Executed only on the server side
- requires more resources compared with SPA
- e.g. MPA (amazon.in)

Single page application (SPA)

- the application which has only single html page
- Gets loaded only once when user visits the website
- Once loaded, it sends the request to the server only to get the data
- Executed only on the client side (inside a browser)
- requires less resources compared with MPA
- it has only one html page named index.html
- e.g. react, angular, vue

Advantages of SPA

- SPA application can cache the contents for offline use
- faster than multi-page application
- provides component-based architecture which promotes the re-usability

Disadvantages of SPA

- the first time loading is bit slower than multi-page application
- it has to load all the components(sections) while loading the application for the first time
- these applications are memory-heavy

Frameworks or libraries used to develop SPAs

1. react: developed by Facebook in 2013
2. angular: developed by Google in 2011
3. preact: light weight react
4. svelte: faster than react but has less ecosystem

tooling for any framework

- language: react (JS), angular (TS), preact (JS), svelte (JS)
- bundler:
  - tool to create a bundle (archive of all the required pages of a web application)
  - e.g. Webpack, vite
- platform: node.js

SUNBEAM INFOTECH

Angular

- angular is a framework used to develop SPA type application
- developed by Google in 2011
- developed in TypeScript
- TS is required to develop angular application
- bit slower than react as it requires more memory
- framework
    - contains different modules/libraries
    - most of the times, a frame is built using multiple languages
    - e.g. angular: TS + HTML + CSS + JS + others

What is React

- It is a JS library used to develop Single Page Application
- React is used to develop client side applications
- React features
    - Has a component-driven architecture
    - Uses virtual DOM for improving the application performance
    - Eco-system: React Router, React Redux Toolkit, React Native
- since it is a library, it requires less memory compared to Angular
- it is bit faster than Angular

History

- Created by: Jordan Walke at Facebook (2011)
- First used: Facebook News Feed (2011)
- Open-sourced: May 2013 (React Conf 2015 marked major adoption)
- Current maintainers: Meta (Facebook) + community (OpenJS Foundation)

Virtual DOM

- in-memory copy of browser or real DOM
- real DOM is created by the browser to render the UI in the browser's window
- when real DOM changes, the entire real DOM gets re-rendered
- this is very memory-heavy operation which may slow down the UI generation
- react create a in-memory of real DOM called as virtual DOM to solve the re-rendering of web page when there is a change in the page
- how virtual DOM works
    - when react application loads, react library creates a copy of real DOM and renders the entire web page when it gets loaded initially (only for the first time)
    - when there is a change required, the change will be performed on the virtual DOM first
    - it will updates the virtual DOM with new copy of changed element
    - then react finds the difference between old DOM object and new DOM object (diffing algorithm)
    - then react creates a patch of new changes to be applied on the real DOM
    - finally the patch will be applied on real DOM to change the browser UI (re-concilation)
    - because of this process, the entire real DOM never reloads which is why the react application is faster than MPA

SUNBEAM INFOTECH

CDN links used for react

```
<script src="https://unpkg.com/react@18/umd/react.development.js"></script>
<script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js">
</script>
<!-- Don't use this in production: -->
<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
```

- `react.development.js`
  - used for developing the react application
  - used to create react components
- `react-dom.development.js`
  - used to render the react components inside the browser
- `babel.min.js`
  - used to convert the JSX code into JS code

Pure React (Without JSX)

Core Concepts

- Direct manipulation of Virtual DOM using React's JavaScript API
- Uses `React.createElement()` to construct UI elements
- Manual DOM mounting with `ReactDOM.createRoot()`

Example

```
<!DOCTYPE html>
<html>
    <head>
        <script
            crossorigin
            src="https://unpkg.com/react@18/umd/react.development.js"
        ></script>
        <script
            crossorigin
            src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
        ></script>
    </head>
    <body>
        <div id="root1"></div>
        <div id="root2"></div>

        <script>
            // Simple element creation
            const root1 = document.getElementById("root1");
            const h1 = React.createElement("h1", null, "Hello React");
            ReactDOM.render(h1,root1); //(prior react 18)

            // Complex element hierarchy
```

SUNBEAM INFOTECH

```
            const root2 = ReactDOM.createRoot(document.getElementById("root2"));
            const h4 = React.createElement(
                "h4",
                { key: "h4" },
                "The library for web and native user interfaces"
            );
            const p = React.createElement(
                "p",
                { key: "p" },
                "React is a free and open-source front-end JavaScript library that
aims to make building user interfaces based on components more seamless."
            );
            const container = React.createElement("div", null, [h4, p]);
            root2.render(container); //(from react 18)
        </script>
    </body>
</html>
```

Key Observations in above example

1. Verbose Syntax: Each element requires explicit `createElement` call
2. Manual Structure: Nested elements become hard to visualize
3. No Components: Just direct Virtual DOM node creation
4. Performance: Still uses Virtual DOM diffing

Render single element(From React 18)

```
const root = document.getElementById('root')
const element = React.createElement('h2', {}, 'welcome to react')
ReactDOM.createRoot(root).render(element)
```

Render multiple elements

```
 // get root element
 const root = document.getElementById('root')
// create required element(s)
 const h1 = React.createElement('h1', {}, 'this is a header1')
const h2 = React.createElement('h2', {}, 'this is a header2')
const h3 = React.createElement('h3', {}, 'this is a header3')
// render the elements by using an array of elements
ReactDOM.createRoot(root).render([h1, h2, h3])
```

Render hierarchy of element

```
// create root element
 const root = document.getElementById('root')
```

SUNBEAM INFOTECH

```
// create a child element
 const h1 = React.createElement('h1', {}, 'Child element of div')
// create parent div
 const div = React.createElement('div', {}, h1)
// render the element
ReactDOM.createRoot(root).render(div)
```

React (Using Babel + JSX)

Babel's Role

- It is a JavaScript compiler used to compile one form of JS to another
- Converts modern JS/JSX to browser-compatible code
- used to convert higher version (which browser can NOT understand) of JS to lower version (which browser can understand)
- JSX Transformation: Converts HTML-like syntax to `React.createElement()` calls
- can be added in a page using CDN link

Example

```
<!DOCTYPE html>
<html>
    <head>
        <script
            crossorigin
            src="https://unpkg.com/react@18/umd/react.development.js"
        ></script>
        <script
            crossorigin
            src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
        ></script>
        <script
            crossorigin
            src="https://unpkg.com/@babel/standalone/babel.min.js"
        ></script>
    </head>
    <body>
        <div id="root1"></div>
        <div id="root2"></div>

        <!-- Babel-processed scripts -->
        <script type="text/babel">
            // Component-based approach
            const root1 = ReactDOM.createRoot(document.getElementById("root1"));

            const Hdr1 = () => {
                return <h1>Hello React</h1>;
            };

            root1.render(<Hdr1 />);
```

SUNBEAM INFOTECH

```
        </script>

        <script type="text/babel">
            const root2 = ReactDOM.createRoot(document.getElementById("root2"));

            const Container = () => {
                return (
                    <div>
                        <h4>The library for web and native user interfaces</h4>
                        <p>
                            React is a free and open-source front-end JavaScript
library that
                            aims to make building user interfaces based on
components more
                            seamless.
                        </p>
                    </div>
                );
            };

            root2.render(<Container />);
        </script>
    </body>
</html>
```

Key Improvements

1. JSX Syntax: HTML-like templating

```
// Transforms to:
// React.createElement("h1", null, "Hello React")
<h1>Hello React</h1>
```

2. Component Model: Reusable UI units

```
function MyComponent() {
    return <div>Content</div>;
}
```

3. Readability: Visual hierarchy matches DOM structure

4. Developer Experience: Faster prototyping

JSX Under the Hood

Compilation Process

- Test on Babel website (with classic runtime).

SUNBEAM INFOTECH

- https://babeljs.io/repl

```
// Original JSX
<div className="header">
    <h1>Title</h1>
    <p>Content</p>
</div>;

// compiled to:
React.createElement(
    "div",
    { className: "header" },
    // next arguments are variable arg list that is collected as array [...]
    React.createElement("h1", null, "Title"),
    React.createElement("p", null, "Content")
);
```

Vite

- a build tool used to build multiple types of project
- e.g. react, angular, svelte, preact etc.
- build
  - compile the project: minifying (reducing the file size) the js files
  - add the external referenced packages
  - create a bundle (collection of required files) for deployment
- create a vite project using npm

project management

```
# create a new project
# > npm create vite <application name>
> npm create vite app1

# go to the project
> cd app1

# install the dependencies
> npm install

# start running the application
> npm run dev
```

Project hierarchy

- node_modules (directory)
  - contains all the dependencies mentioned in package.json
- public (directory)

SUNBEAM INFOTECH

    - contains the public resources like images, audio, video etc.
- src (directory)
    - contains the application source code (components, assets etc)
    - assets (directory)
        - contains application assets like images
    - components (directory)
        - contains the reusable components
    - pages (directory)
        - contains the components which represent the website pages
    - services (directory)
        - contains the functions which connects the frontend to the backend
        - the function will call the REST APIs
    - App.css
        - contains the css rules for App component
    - App.jsx
        - contains the application's startup or main component
        - the application starts with App component
    - index.css
        - contains the css rules shared across the components
    - main.jsx
        - contains the application startup code
- .gitignore
    - used to mention the files which need not to be added to the git repository
- eslint.config.js
    - configuration file used by ESLint
- index.html
    - only html file in the project
    - contains the application startup code
- package.json
    - contains the application dependencies, scripts and metadata
    - dependencies
        - list of modules required to run the application
        - these modules will be bundled when the app gets built
    - devDependencies
        - list of modules required to develop the application
        - these modules will NOT be bundled when the application gets built
- Readme.md
    - contains the information about the application
- vite.config.js
    - contains the vite configuration
- yarn.lock
    - contains the packages installed with the required versions
    - created when yarn is used to install the dependencies
- package-lock.json
    - contains the packages installed with the required versions
    - created when npm is used to install the dependencies

SUNBEAM INFOTECH

Project startup

- yarn dev or npm run dev is fired
- starts lite web server on port 5173
- the server loads the file named index.html
- index.html loads src/main.jsx
- main.jsx
    - find an object of div having root id
    - renders the first component named App (which is defined in src/App.jsx)

SUNBEAM INFOTECH